Corrigé du TP

Sécurisation de l'API Planora

Document enseignant

TP réalisé par :
Heitz Romaric, Khiri Amine, Benali Syrine,
Van-Rossen-Martinez Alessandra et Denocq Elio

Table des matières

1 Exercice 1 — Enquête sur le contrôle d'accès **2**
1.1 Résultats observés dans `Tp_version1` . . . 2
1.2 Réponses aux questions . . . 2

2 Exercice 2 — Ajouter Spring Security **2**
2.1 TODO 2.1 — Dépendance Maven . . . 2
2.2 TODO 2.2 à 2.4 — Comportement attendu . . . 3
2.3 TODO 2.5 — Test . . . 3
2.4 Réponses aux questions . . . 3

3 Exercice 3 — Comptes et BCrypt **3**
3.1 TODO 3.1 à 3.4 — Fichier `UserConfig.java` . . . 3
3.2 TODO 3.5 — Test et résultats . . . 4
3.3 Réponses aux questions . . . 4

4 Exercice 4 — Autorisations HTTP **5**
4.1 TODO 4.1 à 4.6 — `SecurityConfig.java` . . . 5
4.2 Résultats attendus . . . 6
4.3 TODO 4.7 et 4.8 — Vérification . . . 6
4.4 Réponses aux questions . . . 6

5 Exercice 5 — Protection d'une méthode **6**
5.1 État initial . . . 6
5.2 TODO 5.1 — `@PreAuthorize` . . . 7
5.3 TODO 5.2 — `@EnableMethodSecurity` . . . 7
5.4 TODO 5.3 — Test et résultats . . . 7
5.5 Réponses aux questions . . . 7

6 Exercice 6 — Vérification finale **8**
6.1 Matrice finale . . . 8
6.2 Réponses au bilan . . . 8

1 Exercice 1 — Enquête sur le contrôle d'accès

1.1 Résultats observés dans `Tp_version1`

Toutes les opérations métier sont accessibles sans compte. Pour les requêtes valides et les identifiants de tâche existants, les réponses attendues sont :

Requête	Statut	Résultat
`GET /api/public/info`	200	Message d'information publique
`GET /api/tasks`	200	Liste des tâches
`POST /api/tasks`	201	Tâche créée
`PUT /api/tasks/1`	200	Tâche modifiée
`PATCH /api/tasks/1/complete`	200	Tâche marquée terminée
`DELETE /api/tasks/id`	204	Tâche supprimée, sans corps de réponse
`GET /api/admin/stats`	200	Statistiques de l'équipe

Une entrée invalide peut renvoyer `400`, un identifiant inexistant `404`. Pour tester DELETE sans détruire une donnée d'exemple, créer une tâche pendant l'exercice puis supprimer cette nouvelle tâche.

1.2 Réponses aux questions

1. **Permissions attendues.** Une personne anonyme ne devrait pouvoir accéder qu'à l'information explicitement publique. Un membre authentifié peut consulter et gérer les tâches partagées de l'équipe. Un administrateur peut en plus consulter les statistiques administratives et, à partir de l'exercice 5, le projet confidentiel. Dans ce TP, les tâches sont partagées : aucune règle de propriété individuelle n'est prévue.
2. **Lien avec OWASP A01 :2025 — Broken Access Control.** Le serveur n'applique pas de contrôle d'accès aux routes métier. Un visiteur peut lire les tâches, les modifier ou les supprimer et consulter les statistiques. Par exemple, `DELETE /api/tasks/{id}` est accepté sans authentification ; `GET /api/admin/stats` révèle des indicateurs à un visiteur. Cela correspond à un contrôle d'accès manquant. Le nom `/admin/` dans une URL n'applique aucune permission par lui-même.
   Référence : https://top10.owasp.org/2025/A01_2025-Broken_Access_Control/.
3. **Risques.** Un visiteur peut accéder à des données de l'équipe, modifier les tâches ou effacer le travail existant. Il peut aussi consulter des informations d'administration. L'impact dépend des données et opérations réellement exposées.
4. **Authentification et autorisation.** L'authentification vérifie l'identité présentée. L'autorisation décide si cette identité possède le droit d'effectuer une action donnée.
5. **Emplacement du contrôle.** Masquer un bouton dans React ne bloque pas une requête envoyée directement à l'API avec curl ou un autre client. Les contrôles doivent être appliqués dans le backend Spring, qui reçoit toutes les requêtes et peut imposer les permissions indépendamment de l'interface.

2 Exercice 2 — Ajouter Spring Security

2.1 TODO 2.1 — Dépendance Maven

Ajouter dans `pom.xml`, à l'emplacement indiqué :

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
```

```
</dependency>
```

Ne pas renseigner de version : le parent Spring Boot gère les versions des dépendances.

2.2 TODO 2.2 à 2.4 — Comportement attendu

L'ajout du starter déclenche la configuration par défaut de Spring Security. Tant qu'aucun `UserDetailsService` personnalisé n'est fourni, Spring crée l'utilisateur `user` et affiche un mot de passe temporaire dans la console au démarrage.

- Une requête GET anonyme vers `/api/public/info`, `/api/tasks` ou `/api/admin/stats` reçoit `401 Unauthorized`. Par défaut, toutes les routes sont protégées ; aucune exception publique n'a encore été configurée.
- Une lecture avec `user` et le mot de passe affiché reçoit `200`.
- Les rôles USER et ADMIN ne sont pas encore distingués. Un utilisateur authentifié peut lire les tâches et les statistiques.
- Les mutations peuvent recevoir `403 Forbidden` faute de jeton CSRF. La protection CSRF est active par défaut et le client fourni n'envoie pas ce jeton. Ce refus n'indique pas un rôle insuffisant.

2.3 TODO 2.5 — Test

Dans `TaskApiSecurityTest.java`, retirer `@Disabled` au-dessus de la méthode suivante :

```
exercice2LesRoutesDemandentUneConnexion
```

Lancer ensuite `mvn test`. Le test attend `401` pour les trois lectures anonymes ; il ne couvre pas les mutations.

2.4 Réponses aux questions

1. Le starter Spring Security et sa configuration automatique créent le compte temporaire lorsque l'application n'a pas de source d'utilisateurs personnalisée.
2. Non. À ce stade, aucune règle d'autorisation par rôle n'est configurée.
3. Le mot « public » dans le nom de la route est une convention de nommage ; Spring n'en déduit pas une permission. Il faut une règle explicite comme `permitAll()`.
4. La lecture GET ne requiert pas de jeton CSRF, tandis qu'une requête qui modifie l'état en requiert un par défaut. Le client de TP ne le transmet pas ; Spring rejette donc la mutation avec `403`.

3 Exercice 3 — Comptes et BCrypt

3.1 TODO 3.1 à 3.4 — Fichier `UserConfig.java`

La classe complète peut être écrite ainsi :

```java
package fr.tpspringsecurity.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
```

```java
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.provisioning.InMemoryUserDetailsManager;

@Configuration
public class UserConfig {

    // TODO 3.1 : fournir l'encodeur utilise par Spring Security.
    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    // TODO 3.2 : declarer la source des comptes en memoire.
    @Bean
    public UserDetailsService userDetailsService(PasswordEncoder encoder) {
        // TODO 3.3 : construire Alice et Bob, avec mots de passe encodes.
        UserDetails alice = User.withUsername("alice")
                .password(encoder.encode("alice123"))
                .roles("USER")
                .build();

        UserDetails bob = User.withUsername("bob")
                .password(encoder.encode("bob123"))
                .roles("ADMIN")
                .build();

        // TODO 3.4 : fournir les deux utilisateurs au gestionnaire en memoire.
        return new InMemoryUserDetailsManager(alice, bob);
    }
}
```

TODO 3.1 Le Bean `PasswordEncoder` retourne un `BCryptPasswordEncoder`.

TODO 3.2 Le Bean `UserDetailsService` reçoit cet encodeur.

TODO 3.3 Alice a le rôle `USER` et Bob le rôle `ADMIN`. Les deux mots de passe sont encodés avant stockage.

TODO 3.4 Le `InMemoryUserDetailsManager` contient les deux utilisateurs et remplace l'utilisateur temporaire.

Spring ajoute le préfixe `ROLE_` aux rôles déclarés par `.roles(...)`. Ainsi, `.roles("ADMIN")` donne l'autorité `ROLE_ADMIN`, recherchée ultérieurement par `hasRole("ADMIN")`.

3.2 TODO 3.5 — Test et résultats

Retirer `@Disabled` du test `exercice3AliceEtBobPeuventSeConnecter`, puis lancer `mvn test`. Le test vérifie que les GET des tâches fonctionnent avec les deux comptes et qu'un mauvais mot de passe produit `401`. Vérifier séparément dans Planora que les deux comptes accèdent aux statistiques.

- Alice : GET tâches `200`, GET statistiques `200`.
- Bob : GET tâches `200`, GET statistiques `200`.
- Mauvais mot de passe : `401`.
- Mutation sans jeton CSRF : peut être refusée avec `403`, avec ou sans compte.

3.3 Réponses aux questions

BCrypt transforme le mot de passe en une empreinte non réversible avant son stockage. Lors d'une connexion, Spring compare le mot de passe présenté à l'empreinte ; il ne le déchiffre pas. Les

mots de passe en clair présents dans ce TP sont uniquement des identifiants de démonstration.

Alice peut lire les statistiques car son rôle est seulement déclaré ; aucune règle d'accès ne le vérifie encore. Authentifier Alice confirme son identité ; autoriser Alice à consulter une ressource dépend d'une règle qui compare ses droits à l'action demandée.

4 Exercice 4 — Autorisations HTTP

4.1 TODO 4.1 à 4.6 — `SecurityConfig.java`

Configuration de la chaîne HTTP à réaliser en version 4 :

```java
package fr.tpspringsecurity.config;

import static org.springframework.security.config.Customizer.withDefaults;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.
    EnableWebSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
            throws Exception {
        return http
            // TODO 4.5 : chaque requete fournit ses identifiants.
            .sessionManagement(session -> session
                .sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            // TODO 4.6 : choix du TP pour le client React fourni.
            .csrf(csrf -> csrf.disable())
            .authorizeHttpRequests(auth -> auth
                // TODO 4.2 : information publique.
                .requestMatchers(HttpMethod.GET, "/api/public/info")
                    .permitAll()
                // TODO 4.3 : statistiques reservees a ADMIN.
                .requestMatchers(HttpMethod.GET, "/api/admin/**")
                    .hasRole("ADMIN")
                // TODO 4.4 : toutes les autres requetes sont authentifiees.
                .anyRequest().authenticated())
            // TODO 4.5 : activer HTTP Basic.
            .httpBasic(withDefaults())
            .build();
    }
}
```

TODO 4.1 Déclarer un Bean `SecurityFilterChain` qui reçoit `HttpSecurity` et retourne la chaîne construite.

TODO 4.2 Autoriser explicitement le GET de l'information publique avec `permitAll()`.

TODO 4.3 Réserver les GET sous `/api/admin/` au rôle ADMIN avec `hasRole("ADMIN")`.

TODO 4.4 Exiger une identité pour les autres routes. Cette règle générale doit rester après les règles plus précises.

TODO 4.5 Activer HTTP Basic et choisir `SessionCreationPolicy.STATELESS`.

TODO 4.6 Désactiver CSRF pour le client local du TP, qui transmet explicitement l'en-tête `Authorization`, n'utilise pas de cookie et envoie les requêtes avec `credentials: "omit"`.

Note sur CSRF. Ce choix est propre au client fourni. HTTP Basic et une politique sans session ne protègent pas automatiquement tous les clients contre CSRF : certains navigateurs peuvent envoyer automatiquement des identifiants Basic. Si l'authentification ou le client change, il faut réévaluer cette décision. La documentation Spring Security rappelle que les applications utilisant HTTP Basic peuvent être vulnérables à CSRF : https://docs.spring.io/spring-security/reference/features/exploits/csrf.html.

4.2 Résultats attendus

Route	Anonyme	Alice USER	Bob ADMIN
GET /api/public/info	200	200	200
GET /api/tasks	401	200	200
POST /api/tasks	401	201	201
GET /api/admin/stats	401	403	200
GET /api/tasks/confidential	401	404	404

La route confidentielle n'existe pas encore en version 4. Avec un compte valide, le filtre laisse passer la requête, puis l'absence de route produit `404`. Un visiteur est bloqué auparavant avec `401`.

Pour les opérations CRUD valides, Alice et Bob peuvent créer (`201`), modifier et terminer (`200`), puis supprimer (`204`). Les statistiques sont interdites à Alice (`403`) et accessibles à Bob (`200`).

4.3 TODO 4.7 et 4.8 — Vérification

Le test `exercice4LAccesPublicEtLesDroitsAdminSontRespectes` vérifie la route publique, le refus des statistiques à Alice, leur accès à Bob et une création authentifiée par Alice. Retirer `@Disabled`, puis exécuter `mvn test`. Si le POST reçoit `403`, vérifier le TODO CSRF.

4.4 Réponses aux questions

Les matchers sont évalués dans l'ordre déclaré. Si `anyRequest()` précédait les règles précises, la configuration serait incorrecte ou les règles suivantes ne pourraient plus affiner la décision. Alice est authentifiée mais son rôle ne correspond pas à `ADMIN`, donc Spring répond `403`. `STATELESS` évite de garder une session de connexion ; le client transmet son en-tête d'authentification à chaque appel. La désactivation CSRF facilite les mutations pour ce client particulier, mais doit être réévaluée si le mode d'authentification ou de transmission change.

5 Exercice 5 — Protection d'une méthode

5.1 État initial

La méthode et la route `GET /api/tasks/confidential` existent déjà en version 5. Elles retournent un objet `ConfidentialProject` par l'intermédiaire de `taskService.confidential()`. La règle HTTP générale exige seulement une authentification : Alice et Bob reçoivent donc tous deux `200` avant la correction ; un visiteur reçoit `401`.

5.2 TODO 5.1 — `@PreAuthorize`

Dans `TaskController.java`, importer `PreAuthorize` et activer l'annotation sur la méthode existante :

```java
import org.springframework.security.access.prepost.PreAuthorize;

// Dans TaskController, sur la methode confidential :
@GetMapping("/confidential")
@PreAuthorize("hasRole('ADMIN')")
public ConfidentialProject confidential() {
    return taskService.confidential();
}
```

Ne pas recréer la route ou déplacer les données dans le contrôleur. L'appel au service reste inchangé.

5.3 TODO 5.2 — `@EnableMethodSecurity`

Dans `SecurityConfig.java`, importer l'annotation et l'ajouter à la classe :

```java
import org.springframework.security.config.annotation.method.configuration.
    EnableMethodSecurity;

@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class SecurityConfig {
    // Le bean SecurityFilterChain existant ne change pas.
}
```

`@EnableMethodSecurity` active les contrôles d'annotations de méthode, dont `@PreAuthorize`. Sans cette activation, la règle placée sur la méthode n'est pas évaluée.

5.4 TODO 5.3 — Test et résultats

Retirer `@Disabled` du test `exercice5MethodeConfidentielleReserveeAuxAdmins` puis lancer `mvn test`. Le test attend :

Profil	Statut	Explication
Anonyme	401	La règle HTTP exige une authentification
Alice USER	403	Identité reconnue, rôle insuffisant
Bob ADMIN	200	Autorisation accordée

5.5 Réponses aux questions

Avant la correction, la règle HTTP générale ne vérifie que l'authentification. Alice peut donc lire la note.

`@EnableMethodSecurity` active l'évaluation des annotations de méthode.

`SecurityFilterChain` contrôle l'accès selon la route HTTP.

`@PreAuthorize` vérifie le rôle juste avant l'exécution de la méthode.

Le contrôleur délègue l'accès aux données au service, qui porte la logique métier.

6 Exercice 6 — Vérification finale

6.1 Matrice finale

Opération	Anonyme	Alice USER	Bob ADMIN
`GET /api/public/info`	200	200	200
`GET /api/tasks`	401	200	200
`POST /api/tasks`	401	201	201
`PUT /api/tasks/{id}`	401	200	200
`PATCH /api/tasks/{id}/complete`	401	200	200
`DELETE /api/tasks/{id}`	401	204	204
`GET /api/admin/stats`	401	403	200
`GET /api/tasks/confidential`	401	403	200

Les codes `401` et `403` ne sont pas interchangeables : `401` signifie que l'authentification manque ou échoue ; `403` signifie que l'identité est reconnue, mais que son autorisation est insuffisante. `201` indique une création réussie ; `204` une suppression réussie sans corps de réponse.

6.2 Réponses au bilan

1. L'exercice 2 active Spring Security et l'authentification par défaut. L'exercice 3 configure Alice et Bob.
2. L'exercice 4 ajoute les autorisations HTTP par route et par rôle.
3. L'exercice 5 active la sécurité de méthode avec `@EnableMethodSecurity` et réserve la méthode confidentielle à ADMIN avec `@PreAuthorize`.
4. Stocker un mot de passe en clair l'expose directement en cas d'accès aux données. BCrypt stocke une empreinte adaptative non réversible.
5. HTTP Basic encode les identifiants en Base64, mais ne chiffre pas le transport. Cette démonstration utilise des comptes faibles et des données fictives ; elle exige HTTPS et une conception adaptée avant tout usage réel.